

# rdd\_practice

February 22, 2026

```
[1]: from pyspark.sql import SparkSession

spark=SparkSession.builder.appName("New Assignment").getOrCreate()

sc=spark.sparkContext

[2]: # rdd qsns
# load data

# load all csvs using SparkContext
cust=sc.textFile("customers.csv")
ord=sc.textFile("orders.csv")
prod=sc.textFile("products.csv")
payments=sc.textFile("payments.csv")

# count total customers
h_c=cust.first()
cust_cleaned=cust.filter(lambda x:x!=h_c).map(lambda x:x.split(", "))
# print(cust_cleaned.collect())
cust_cleaned.count()

# display premium customers from hyd
cust_cleaned.filter(lambda x:x[3]=='Hyderabad' and x[5]=='Premium').collect()

h_o=ord.first()
h_p=prod.first()
h_pay=payments.first()
ord_cleaned=ord.filter(lambda x:x!=h_o).map(lambda x:x.split(", "))
prod_cleaned=prod.filter(lambda x:x!=h_p).map(lambda x:x.split(", "))
payments_cleaned=payments.filter(lambda x:x!=h_pay).map(lambda x:x.split(", "))

# count total number of orders
ord_cleaned.count()

# find distinct payment modes
ord_cleaned.map(lambda x:x[5]).distinct().collect()
```

```
[2]: ['Card', 'Cash', 'UPI']
```

```
[3]: # INTERMEDIATE

# Total quantity sold per product.
total_qty = ord_cleaned.map(lambda x: (x[2], int(x[3]))).reduceByKey(lambda x, y: x + y)
total_qty.collect()
```

```
[3]: [('P001', 2),
      ('P002', 1),
      ('P007', 2),
      ('P006', 7),
      ('P005', 2),
      ('P009', 1),
      ('P004', 3),
      ('P003', 3),
      ('P008', 1),
      ('P010', 1)]
```

```
[4]: # Total revenue per product (join orders + products). revenue --> price*qty,
      ↪ successful revenue --> amount paid(from payments)
# (prod id,price) from products
p1 = prod_cleaned.map(lambda x: (x[0], int(x[4])))
# (prod id,qty) from orders
p2 = ord_cleaned.map(lambda x: (x[2], int(x[3])))

joined = p1.join(p2) # (prodId,(price,qty))
total_revenue = joined.map(lambda x: (x[0], x[1][0] * x[1][1])).reduceByKey(
    lambda x, y: x + y
)
# this can be sorted also
total_revenue.sortBy(lambda x: x[1], ascending=False) # descending
total_revenue.collect()
```

```
[4]: [('P006', 49000),
      ('P007', 120000),
      ('P009', 38000),
      ('P003', 450000),
      ('P004', 120000),
      ('P008', 45000),
      ('P010', 50000),
      ('P001', 160000),
      ('P002', 75000),
      ('P005', 30000)]
```

```
[ ]: # Count customers per state.
per_state=cust_cleaned.map(lambda x: (x[4],1)).reduceByKey(lambda x,y:x+y)
```

```
per_state.collect()
```

```
[ ]: [('MH', 4), ('TN', 2), ('DL', 2), ('TS', 2), ('KA', 2)]
```

```
[ ]: # Find average age per customer_type.  
# to compute age we need sum,count  
avg_age = (  
    cust_cleaned.map(lambda x: (x[5], int(x[6])))  
    .groupByKey()  
    .mapValues(lambda x: sum(x) / len(x))  
)  
avg_age.collect()  
  
# other way using (type,(age,1))  
cust_cleaned.map(lambda x: (x[5], (int(x[6]), 1))).reduceByKey(  
    lambda a, b: (a[0] + b[0], a[1] + b[1])  
)  
.mapValues(lambda x: x[0] / x[1]).collect()
```

```
[ ]: [('Premium', 36.5), ('Regular', 29.333333333333332)]
```

```
[ ]: # Find top 3 most ordered products (by quantity).  
top_3=ord_cleaned.map(lambda x: (x[2],int(x[3])))  
.reduceByKey(lambda x,y:x+y).  
    ↪sortBy(lambda x:x[1],ascending=False)  
top_3.take(3)
```

```
[ ]: [('P006', 7), ('P004', 3), ('P003', 3)]
```

```
[7]: # ADVANCED (MULTI GROUPBY + AGG)  
  
# Total revenue per (state, category).  
  
# (cust_id,state) from customers  
# (prod_id,cust_id,qty) from orders  
# (prod_id,category,price) from products  
  
p1 = ord_cleaned.map(lambda x: (x[2], (x[1], int(x[3])))) #  
    ↪(prodId,(custId,qty))  
p2 = prod_cleaned.map(lambda x: (x[0], (x[2], int(x[4])))) #  
    ↪(prodid,(category,price))  
  
p3 = cust_cleaned.map(lambda x: (x[0], x[4])) # (custid,state)  
  
# join p1 & p2  
joined = p1.join(p2) # (prodid,((custid,qty),(category,price)))  
# now rearrange this to (custid,(category,revenue))  
# joined.collect()
```

```

# now (custId,(category,revenue))
revenue = joined.map(lambda x: (x[1][0][0], # cust id
                                (x[1][1][0], x[1][0][1]*x[1][1][1] #
    ↪ sales*price=revenue
)))
# revenue.collect()

joined2=revenue.join(p3) # (custId,((category,revenue),(state))
# rearrange to groupby (state,category)
final_prep=joined2.map(lambda x:(
    (x[1][1],x[1][0][0]), #(state,category)
    x[1][0][1] # revenue
))

res=final_prep.reduceByKey(lambda x,y:x+y)
res.collect()
# for (state, category), revenue in res.sortByKey().collect():
#     print(f"{state} - {category} : {revenue}")

```

```

[7]: [ (('TS', 'Electronics'), 275000),
      (('DL', 'Furniture'), 40000),
      (('MH', 'Furniture'), 101000),
      (('TN', 'Electronics'), 45000),
      (('DL', 'Electronics'), 150000),
      (('TS', 'Furniture'), 30000),
      (('MH', 'Electronics'), 38000),
      (('TN', 'Furniture'), 28000),
      (('KA', 'Electronics'), 380000),
      (('KA', 'Furniture'), 50000)]

```

```

[8]: # For each customer_type, find:
# total orders
# total revenue
# average order value

p1 = cust_cleaned.map(lambda x: (x[0], x[5])) # (custId,type)
p2 = ord_cleaned.map(lambda x: (x[2], (x[1], int(x[3])))) #
    ↪ (prodid,(custid,qty))
p3 = prod_cleaned.map(lambda x: (x[0], int(x[4]))) # (prodid,price)

# revenue per order
# calculate total revenue
joined = p2.join(p3) # (prodId,((custId,qty),price))

# (custId,order_revenue)
revn_per_order = joined.map(lambda x: (x[1][0][0], x[1][0][1] * x[1][1]))

```

```

# join with p1
final_joined = revn_per_order.join(p1) # (custId,(revenue,type))

# now we need (customer_type, (order_count, total_revenue))
prep = final_joined.map(
    lambda x: (x[1][1], (1, x[1][0])) # cust_type # (order_count,revenue)
)

# reduce
res = prep.reduceByKey(
    lambda x, y: (x[0] + y[0], x[1] + y[1]) # total orders # total revenue
)

# now avg
final = res.map(
    lambda x: (
        x[0],
        x[1][0], # total orders
        x[1][1], # total revenue
        x[1][1] / x[1][0], # avg revenue
    )
)

for row in final.collect():
    print(
        f"Type: {row[0]} | Orders: {row[1]} | Revenue: {row[2]} | Avg Order: {
round(row[3],2)}"
    )

```

Type: Regular | Orders: 5 | Revenue: 207000 | Avg Order: 41400.0  
Type: Premium | Orders: 10 | Revenue: 930000 | Avg Order: 93000.0

[11]: # Find customers who made more than 1 order AND total spent > 100000.

```

# (orderId,custId) from orders
orders_small = ord_cleaned.map(lambda x: (x[0], x[1]))

# (orderId, amountPaid) from payments
payments_small = payments_cleaned.map(lambda x: (x[1], int(x[3])))

j1=orders_small.join(payments_small) #(orderId,(custId,amount paid))

# remap
cust_level = j1.map(lambda x: (x[1][0], (1, x[1][1])))
# (custId, (1, amountPaid))

```

```

agg=cust_level.reduceByKey(lambda x,y:(x[0]+y[0],x[1]+y[1])) #
    ↪(custId,(total_orders,total_spent))

cust_names=cust_cleaned.map(lambda x:(x[0],x[1]+" "+x[2]))

filtered=agg.filter(lambda x:x[1][0]>1 and x[1][1]>10000)
final=filtered.join(cust_names)

res=final.map(lambda x:(
    x[1][1], # name
    x[1][0][0], # total orders
    x[1][0][1] # total spent
))
res.take(3)

```

```

[11]: [('Ravi Kumar', 2, 155000),
      ('David Paul', 2, 150000),
      ('John Mathew', 2, 200000)]

```

```

[ ]: # Find failed payment revenue loss per category.
# (orderId, amount) where payment failed
failed_pay = payments_cleaned.filter(lambda x: x[4] == "Failed").map(
    lambda x: (x[1], int(x[3])))
)

# (orderId, productId)
orders_small = ord_cleaned.map(lambda x: (x[0], x[2]))

# join failed payments with orders
j1 = failed_pay.join(orders_small)
# (orderId, (amount, productId))

# (productId, amount)
prod_level = j1.map(lambda x: (x[1][1], x[1][0]))

# (productId, category)
products_small = prod_cleaned.map(lambda x: (x[0], x[2]))

# join with products
j2 = prod_level.join(products_small)
# (productId, (amount, category))

# (category, amount)
category_loss = j2.map(lambda x: (x[1][1], x[1][0])).reduceByKey(lambda x, y: x
    ↪+ y)

print("Failed Revenue Loss Per Category:")

```

```
category_loss.take(4)
```

```
[16]: # For each state, find top spending customer.
# (orderId, custId)
orders_small2 = ord_cleaned.map(lambda x: (x[0], x[1]))

# (orderId, amount)
payments_small = payments_cleaned.map(lambda x: (x[1], int(x[3])))

# join orders + payments
j3 = orders_small2.join(payments_small)
# (orderId, (custId, amount))

# (custId, total_spent)
cust_spent = j3.map(lambda x: (x[1][0], x[1][1])).reduceByKey(lambda x, y: x + y)

# (custId, state)
cust_state = cust_cleaned.map(lambda x: (x[0], x[4]))

# join to get state
j4 = cust_spent.join(cust_state)
# (custId, (total_spent, state))

# (state, (custId, total_spent))
state_level = j4.map(lambda x: (x[1][1], (x[0], x[1][0])))

# get max spender per state
top_customer = state_level.reduceByKey(lambda x, y: x if x[1] > y[1] else y)

print("Top Spending Customer Per State:")
top_customer.collect()
```

Top Spending Customer Per State:

```
[16]: [('TN', ('C009', 45000)),
      ('DL', ('C011', 150000)),
      ('MH', ('C002', 118000)),
      ('TS', ('C001', 155000)),
      ('KA', ('C007', 230000))]
```

```
[12]: # Customers who never placed orders (ANTI JOIN)
# (custId, name)
cust_ids = cust_cleaned.map(lambda x: (x[0], x[1] + " " + x[2]))

# (custId, 1) from orders
orders_cust = ord_cleaned.map(lambda x: (x[1], 1))
```

```

# LEFT OUTER JOIN
left_join = cust_ids.leftOuterJoin(orders_cust)
# (custId, (name, order_or_None))

# filter where order is None
never_ordered = left_join.filter(lambda x: x[1][1] is None)

print("Customers Who Never Placed Orders:")
never_ordered.collect()

```

Customers Who Never Placed Orders:

```
[12]: [('C010', ('Latha Iyer', None)), ('C006', ('Meena Rao', None))]
```

RDD – 15 Medium/Advanced Questions

(You must use key-value RDD patterns, joins, aggregations, sorting, multi-stage transforms)

1 Revenue Intelligence

For each (state, category) compute:

total successful revenue

total failed revenue

revenue difference Sort by revenue difference descending.

2 Premium Behavior Analysis

For Premium customers:

total orders

total quantity

total revenue

average order value Return top 5 Premium customers by revenue.

3 Multi-Level Aggregation

For each (customer\_type, category) compute:

total revenue

unique customers

total quantity

4 Repeat Purchase Detection

Find customers who:

purchased the same product more than once

AND total quantity > 2

5 State-wise Product Dominance



For each state:

find the most sold category (by quantity)

include total quantity

## 6 Revenue Concentration

Find top 20% customers contributing to revenue. (Hint: total revenue  $\rightarrow$  sort  $\rightarrow$  cumulative sum logic)

## 7 Order Frequency Segmentation

Classify customers:

Low (1 order)

Medium (2–3)

High (4+)

Return count per segment.

## 8 Failed Payment Impact

For each category:

count of failed payments

total failed revenue Sort descending.

## 9 Multi-Join Heavy

Join all 4 datasets and compute: For each (state, payment\_mode):

total revenue

avg order value

unique customers

Cross Category Customers

Find customers who purchased:

Electronics AND Furniture

### 1.1 Anti Join Scenario

Find customers who:

placed orders

but have no successful payments

### 1.2 combineByKey Practice

For each state compute:

min revenue

max revenue

avg revenue

### 1 3 Set Operations

Find:

customers who ordered MacBook but not iPhone

customers who ordered both

### 1 4 Ranking Inside State

For each state:

rank customers by revenue

return top 2

### 1 5 Complex Interview-Level

For each category:

revenue share % of total revenue

cumulative revenue contribution Sort by revenue descending.

DATAFRAME – 15 Medium/Advanced Questions

(Use groupBy, agg, join, window functions, having, subqueries, etc.)

1 Revenue per (state, category) including success & failure separately.

2 Top 3 customers per state by revenue (use window rank).

3 Customers whose average order value is above overall average.

4 Category contribution percentage to total revenue.

5 Revenue growth simulation:

Assume 10% price increase for Electronics. Recompute total revenue.

6 Customers who purchased at least 2 different categories.

7 State-wise highest revenue product.

8 Customers with failed payments more than successful payments.

9 Payment mode performance:

For each payment\_mode:

success rate

total revenue

avg order value

Revenue by age group

Create age groups:

<30

30–40

40+

Compute revenue per group.

1 1 Window Function Heavy

For each category:

rank products by revenue

dense\_rank

row\_number

1 2 Detect Revenue Outliers

Find customers whose revenue > 2x average revenue of their state.

1 3 Running Total Revenue per State (ordered by date)

1 4 Pivot Analysis

Pivot revenue by payment\_mode for each state.

1 5 Complex:

For each customer\_type:

revenue

failed revenue %

avg quantity

highest single order value

SQL – 15 Medium/Advanced Questions

(Register all tables as views.)

1 Revenue per state including success and failure separately.

2 Top 2 customers per state using window functions.

3 Customers whose revenue > average revenue of their state.

4 Revenue contribution % per category.

5 Customers who purchased products from both categories.

6 State-wise best performing product.

7 Payment mode success rate.

8 Customers with no successful payments (anti-join).

9 Revenue by age group with CASE statement.

Rank categories by revenue globally.

1 1 Cumulative revenue contribution per category.

1 2 Customers whose failed payment % > 30%.

1 3 Monthly revenue trend (extract month from date).

1 4 State-wise revenue share over total revenue.

1 5 Hardcore SQL Interview Question:

For each state:

total revenue

avg order value

highest customer revenue

% contribution of top customer

All in single query using subqueries + window functions.